



数据结构与算法分析



人工智能与自动化学院

韩守东

shoudonghan@hust.edu.cn

13720301586



11. 贪心算法

- ▶ 贪心算法的概念
- ▶ 贪心算法的基本要素
 - (1) 贪心选择性质
 - (2) 最优子结构性质
- ▶ 贪心算法是否适用举例
- ▶ 通过例子学习贪心算法策略
 - (1) 活动安排问题
 - (2) 最优装载问题
 - (3) 单源最短路径
 - (4) 多机调度问题



贪心算法基本思想

- ▶ 作出在当前看来最好的选择（比如找钱问题：88元）
 - 非整体考虑，而是某种意义上的局部最优选择
 - 但希望最终达到全局最优
 - 并非所有问题都可得到整体最优解，但对许多问题它能产生（一个）整体最优解，如：单源最短路径问题、最小生成树问题等
 - 在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是最优解的很好近似



活动安排问题-贪心算法的示例

- ▶ **问题简述：**要在所给的活动集合中选出**最大的相容活动子集合**
 - 要求高效地安排一系列争用某一公共资源的活动
 - 贪心算法可有效求解
- ▶ **贪心算法求解：**
 - 使得**尽可能多的活动在更多的剩余占用下能兼容地使用公共资源**
 - 简单、漂亮



活动安排问题

- ▶ **问题定义**: 设有 n 个活动的集合 $E=\{1,2,\dots,n\}$, 其中每个活动都要求使用同一资源, 如演讲会场等, 而在同一时间内只有一个活动能使用这一资源。
 - 每个活动 i 都有一个要求使用该资源的起始时间 s_i 和一个结束时间 f_i , 且 $s_i < f_i$
 - 如果选择了活动 i , 则它在半开时间区间 $[s_i, f_i)$ 内占用资源
 - 若区间 $[s_i, f_i)$ 与区间 $[s_j, f_j)$ 不相交, 则称**活动 i 与活动 j 是相容的**, 即, 当 $s_i \geq f_j$ 或 $s_j \geq f_i$ 时, 活动 i 与活动 j 相容
- ▶ **目标**: 选择最多的互不冲突的活动, 使相容活动集合最大, 找最大相容子集合



最多三个事件



活动安排问题

► 贪心算法实现-GreedySelector

```
GreedySelector :
template<class Type>
void GreedySelector(int n, Type s[], Type f[], bool A[])
{
    A[1]=true;
    int j=1;
    for (int i=2;i<=n;i++) {
        if (s[i]>=f[j]) { A[i]=true; j=i; }
        else A[i]=false;
    }
}
```

各活动的起始时间和
结束时间存储于数组s
和f中且按结束时间的
非减序排列。



活动安排问题

▶ 算法分析

- 由于输入的活动按照其完成时间的非减序排列，所以算法greedySelector每次总是选择具有最早完成时间的相容活动加入集合A中。直观上，按这种方法选择相容活动为未安排活动留下尽可能多的时间。也就是说，该算法的贪心选择的意义是使剩余的可安排时间段极大化，以便安排尽可能多的相容活动
- 算法greedySelector的效率极高
 - ◆ 当输入的活动已按结束时间的非减序排列，算法只需 $O(n)$ 的时间安排n个活动，使最多的活动能相容地使用公共资源
 - ◆ 如果所给出的活动未按非减序排列，可以用 $O(n\log n)$ 的时间重排



活动安排问题

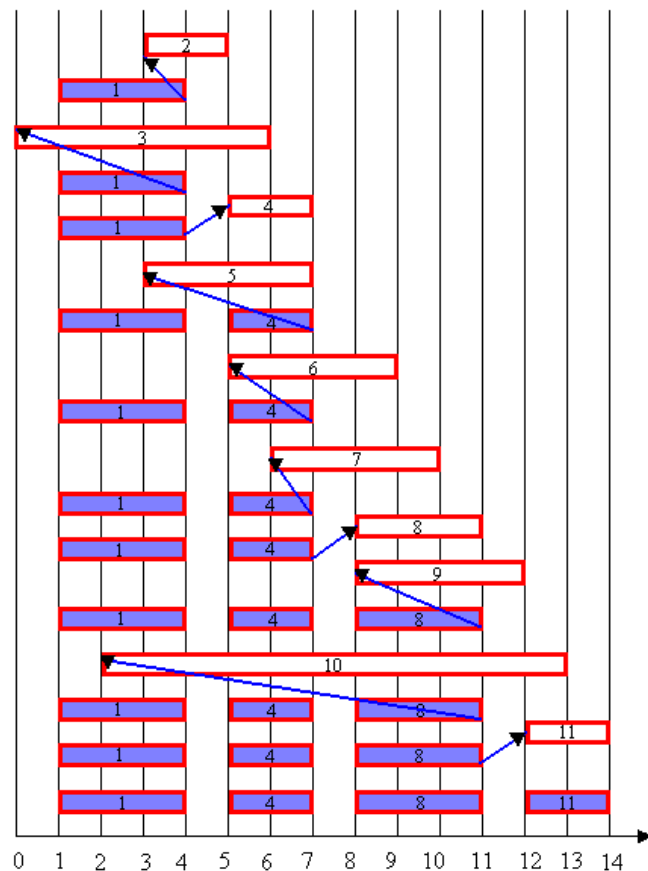
► 例

- 设待安排的11个活动的开始时间和结束时间按结束时间的非减序排列如下：

i	1	2	3	4	5	6	7	8	9	10	11
S[i]	1	3	0	5	3	5	6	8	8	2	12
f[i]	4	5	6	7	8	9	10	11	12	13	14



活动安排问题



➤ 算法greedySelector 的计算过程如左图所示

- 每行相应于算法的一次迭代
- 阴影长条表示的活动是已选入集合A的活动，而空白长条表示的活动是当前正在检查相容性的活动。
- 若被检查的活动i的开始时间 S_i 小于最近选择的活动j的结束时间 f_j ，则不选择活动i，否则选择活动i加入集合A中

贪心算法并不总能求得问题的整体最优解。但对于活动安排问题，贪心算法greedySelector却总能求得整体最优解，即它最终所确定的相容活动集合A的规模最大。这个结论可以用数学归纳法证明。



贪心算法的基本要素



对于一个具体的问题，怎么知道是否可用贪心算法解此问题，以及能否得到问题的最优解呢？

- ▶ 很难给予肯定回答
- ▶ 但可分析问题是否具备两个基本性质
 - 贪心选择性质
 - 最优子结构性质



贪心算法的基本要素

► 贪心选择性质

- 指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素。
- 贪心算法通常以自顶向下的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。

对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所作的贪心选择最终导致问题的整体最优解



贪心算法的基本要素

▶ 最优子结构性质

- 当一个问题 **的最优解包含其子问题的最优解时**，称此问题 **具有最优子结构性质**。

问题的最优子结构性质是该问题可用贪心算法求解的关键特征。



贪心算法是否适用举例：0-1背包问题

问题描述：给定 n 种物品和一个背包。物品 i 的重量是 W_i ，其价值为 V_i ，背包的（负重）容量为 C 。

应如何选择装入背包的物品，使得装入背包中物品的总价值最大？

限制：在选择装入背包的物品时，对每种物品 i 只有2种选择，即装入背包或不装入背包。**不能将物品 i 装入背包多次，也不能只装入物品 i 的一部分。**



贪心算法是否适用举例：背包问题

问题描述： 给定 n 种物品和一个背包。物品 i 的重量是 W_i ，其价值为 V_i ，背包的（负重）容量为 C 。

应如何选择装入背包的物品，使得装入背包中物品的总价值最大？

限制： 可以选择物品 i 的一部分，而不一定要全部装入背包， $1 \leq i \leq n$

这2类问题都具有**最优子结构**性质，极为相似，但背包问题可以用贪心算法求解，而0-1背包问题却不能用贪心算法求解。



0-1 背包问题算法选择分析

- 贪心选择 (V_i / W_i) 之所以不能得到最优解是因为在这种情况下，它无法保证最终能将背包装满，部分闲置的（过大）背包空间使每公斤背包空间的价值降低了。
- 在考虑0-1背包问题时，应比较选择该物品和不选择该物品所导致的最终方案，然后再作出最好选择。如果用贪心算法很容易找出无法得到整体最优解的反例。



贪心算法解背包问题的基本步骤

- 计算每种物品单位重量的价值 V_i / W_i ;
- 依贪心选择策略，将尽可能多的单位重量价值最高的物品装入背包;
- 若将这种物品全部装入背包后，背包内的物品总重量未超过 C ，则选择单位重量价值次高的物品并尽可能多地装入背包;
- 依此策略一直地进行下去，直到背包装满为止（最后选择的物品可能为部分）。



贪心算法解背包问题的算法描述

```
void Knapsack(int n, float M, float v[], float w[], float x[]) {  
    Sort(n, v, w); //将各种物品依其单位重量的价值从大到小排序  
    int i;  
    for (i=1; i<=n; i++) x[i]=0;  
    float c=M;  
    for (i=1; i<=n; i++) {  
        if (w[i]>c) break;  
        x[i]=1;  
        c-=w[i];  
    }  
    if (i<=n) x[i]=c/w[i];  
    //如果有剩余空间, 就将i物品的部分装入。  
}
```

- 算法knapsack的主要计算时间在于将各种物品依其单位重量的价值从大到小排序。因此, 算法的计算时间上界为 $O(n \log n)$ 。
- 为了证明算法的正确性, 还必须证明背包问题具有贪心选择性质。



例1-最优装载

- ▶ **问题描述：** 有一批集装箱要装上一艘载重量为 c 的轮船。其中集装箱 i 的重量为 W_i 。最优装载问题要求确定在装载体积不受限制的情况下，将尽可能多的集装箱装上轮船
- ▶ **算法思想：** 最优装载问题可用贪心算法求解
 - 采用重量最轻者先装的贪心选择策略，可产生最优装载问题的最优解



例1-最优装载

```
template<class Type>
void Loading(int x[],  Type w[], Type c,  int n)
{
    int *t = new int [n+1]; // [0]不用
    Sort(w, t, n); //按重量的升序排序, t(i)=排序i的物品编号
    for (int i = 1; i <= n; i++) x[i] = 0;
    for (int i = 1; i <= n && w[t[i]] <= c; i++)
    {
        x[t[i]] = 1; c -= w[t[i]];
    }
}
```

- 算法loading的主要计算量在于将集装箱依其重量从小到大排序, 故算法所需的计算时间为 $O(n\log n)$



例2. 单源最短路径

► **问题描述：** 给定一个图 $G=(V, E)$ ，其上的边权函数 w 为非负数。给定 V 中的一个顶点，称为**源**。现在要计算从源到所有其它各顶点的**最短路长度**。这里路的长度是指路上各边权之和。这个问题通常称为**单源最短路径问题**。

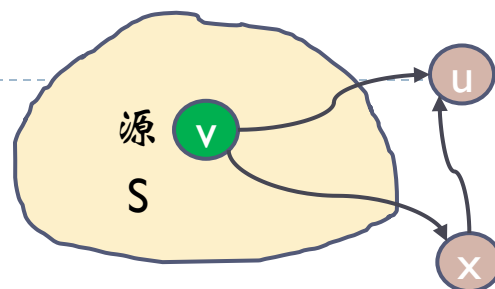
► **算法思想：** **贪心算法求解**

- 设置顶点集合 S 并对其不断扩充，从源点到 S 中顶点的最短距离已知
- 每一步添加 $u \in V - S$ 中具有最小距离的顶点到 S 中
- 更新剩余顶点 u 的距离估计

例2. 单源最短路径

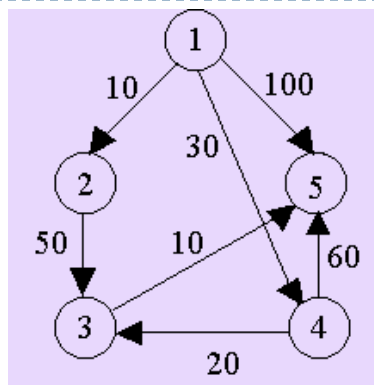
▶ 算法步骤: Dijkstra 贪心算法

- 初始时, S 中仅含有源
- 设 u 是 G 的某一个顶点, 把从源到 u 且中间只经过 S 中顶点的路称为从源到 u 的**特殊路径**, 并用数组 $dist$ 记录当前每个顶点所对应的**最短特殊路径长度**
- 每次从 $V-S$ 中取出具有最短特殊路径长度的顶点 u , 将 u 添加到 S 中, 同时对数组 $dist$ 作必要的修改(只更新 $V-S$)
- 一旦 S 包含了所有 V 中顶点, $dist$ 就记录了从源到所有其它顶点之间的最短路径长度





例2. 单源最短路径



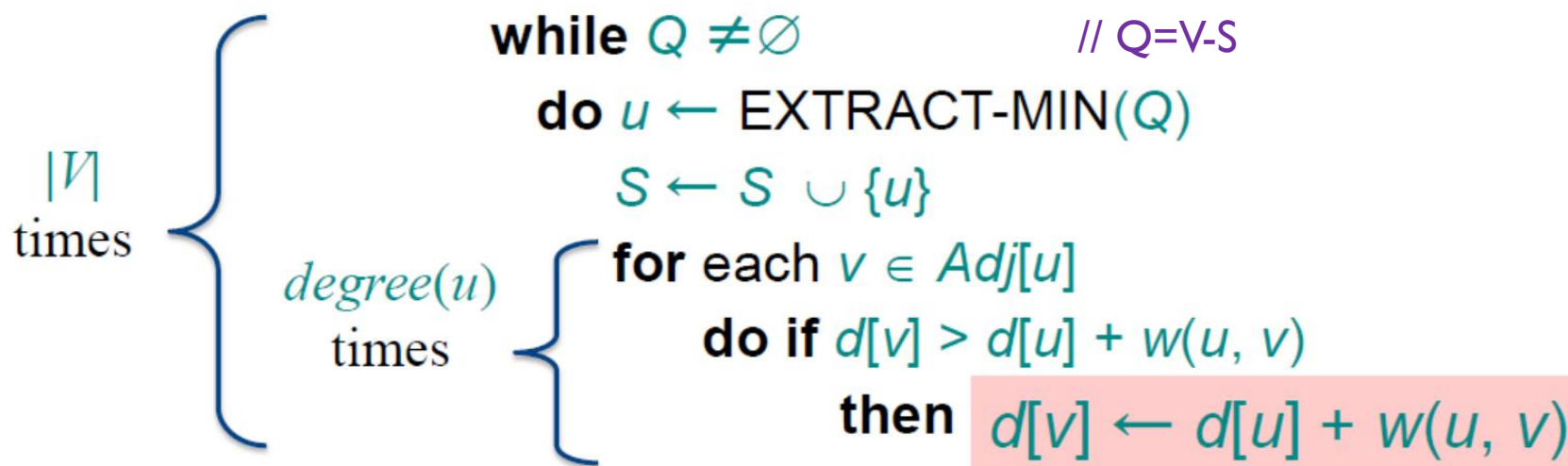
□ 应用Dijkstra算法计算从源顶点1到其它顶点间最短路径的过程

迭代	S	u	dist[2]	dist[3]	dist[4]	dist[5]
初始	{1}	-	10	maxint	30	100
1	{1,2}	2	10	60	30	100
2	{1,2,4}	4	10	50	30	90
3	{1,2,4,3}	3	10	50	30	60
4	{1,2,4,3,5}	5	10	50	30	60



例2. 单源最短路径

- ▶ **计算复杂性：**对于具有 n 个顶点和 e 条边的带权有向图，如果用带权邻接矩阵表示这个图，那么**Dijkstra**算法的主循环体需要 $O(n)$ 时间(找最近邻需 n 次，邻边需 n 次)。这个循环需要执行 $n-1$ 次，所以完成循环需要时间 $O(n^2)$ 。算法的其余部分所需要时间不超过 $O(n^2)$ 。





例3-最小生成树

对于一个无向网络——无向加权连通图 $N=(V,E,C)$ (C 表示该图为权图), 其顶点个数为 $|V|=n$, 图中边的个数为 $|E|$, 我们可以从它的 $|E|$ 条边中选出 $n-1$ 条边, 使之满足

(1) 这 $n-1$ 条边和图的 n 个顶点构成一个连通图。

(2) 该连通图的代价是所有满足条件(1)的连通图的代价的最小值。

这样的连通图被称为网络的最小生成树(Minimum-cost Spanning Tree)。



例3-最小生成树



普里姆(Prim)贪心算法(逐点加入)

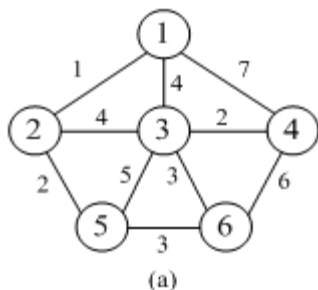
设 $N=(V,E,C)$ 为连通网，**TE** 是 N 的最小生成树 MST 的 **边的集合**，**U** 为 **MST顶点集**。

① 初始设 **$U=\{u_0\}$** ($u_0 \in V$)， **$TE = \Phi$** ；

② 找到满足

$\text{weight}(u,v) = \min\{\text{weight}(u_1,v_1) | u_1 \in U, v_1 \in V-U\}$ 的边，把它**并入TE**，同时 **v 并入U**；

③ 反复执行②，**直至 $V=U$** ，则 **$T=(V,\{TE\})$** 为 N 的**最小生成树**，算法结束。



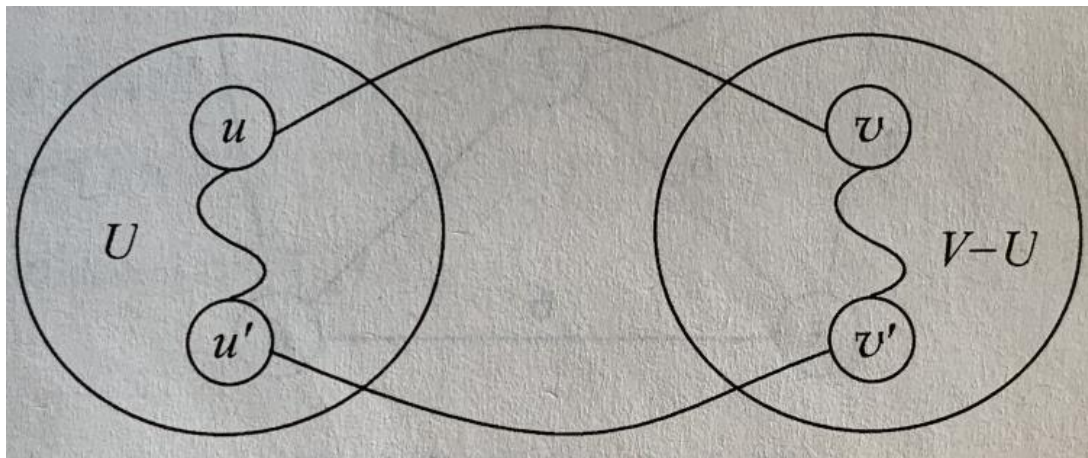


$\begin{array}{c} \text{V} \\ \text{closedge} \end{array}$	2	3	4	5	6	U	V-U
Vex Lowcost	① 1	① 4	① 7	① max	① max	{1}	{2,3,4,5,6}
Vex Lowcost	-1 0	① 4	① 7	② 2	① max	{1,2}	{3,4,5,6}
Vex Lowcost	-1 0	① 4	① 7	-1 0	⑤ 3	{1,2,5}	{3,4,6}
Vex Lowcost	-1 0	⑥ 3	⑥ 6	-1 0	-1 0	{1,2,5,6}	{3,4}
Vex Lowcost	-1 0	-1 0	③ 2	-1 0	-1 0	{1,2,3,5,6}	{4}
Vex Lowcost	-1 0	-1 0	-1 0	-1 0	-1 0	{1,2,3,4,5,6}	$\emptyset$



例3-最小生成树

➤ Prim贪心算法的正确性



MST性质:

设 $G=(V,E)$ 是连通带权图， U 是 V 的真子集。如果 $(u,v) \in E$ ，且 $u \in U, v \in V-U$ ，且在所有这样的边中， (u,v) 的权 $c[u][v]$ 最小，那么一定存在 G 的一颗最小生成树，它以 (u,v) 为其中一条边。

由这一特性，可证明(1)贪心选择性质和(2)最优子结构性质





例4. 多机调度问题

问题描述： 要求给出一种作业调度方案，使所给的 **n 个作业**在尽可能短的时间内由 **m 台机器**加工处理完成。

- 每个作业均可在任何一台机器上加工处理，但未完工前不允许中断处理
- 作业不能拆分成更小的子作业
- 是NP完全问题，到目前为止还没有有效的解法
- 贪心选择策略有时可以设计出较好的近似算法



例4. 多机调度问题

解决思路：采用最长处理时间作业优先的贪心选择策略设计较好的近似算法。

- 当 $n \leq m$ 时，只要将机器 i 的 $[0, t_i]$ 时间区间分配给作业 i 即可，算法只需要 $O(1)$ 时间
- 当 $n > m$ 时，首先将 n 个作业依其所需的处理时间从大到小排序，然后依此顺序将作业分配给最先空闲的处理机
- 算法所需的计算时间为 $O(n \log n)$ （排序消耗的时间）。



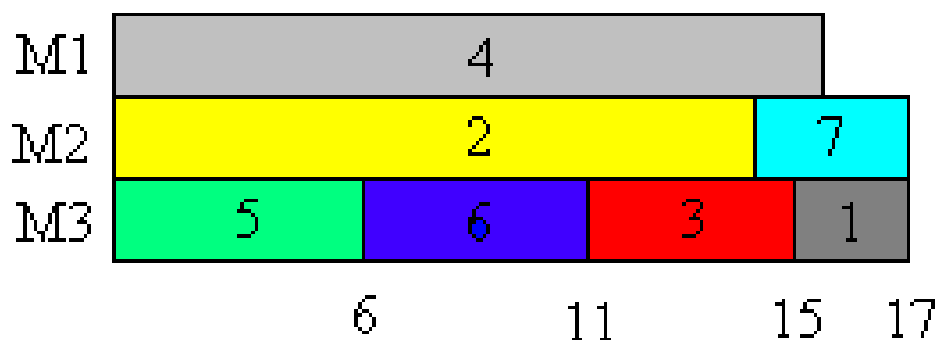
算法实现

```
▶ Template <class type> void Greedy(Type a[], int n, int m){  
    if(n<=m) { // n为作业数, m为机器数  
        cout << “为每个作业分配一台机器” << endl;  
        return; }  
    Sort(a,n); //对n个作业按时间大小从小到大排序 O(nlog(n))  
    MinHeap<MachineNode> H(m); //构建小顶堆动态找m个机器中最先空闲者  
    MachineNode x;  
    for( int i=1; i<=m;i++){  
        x.avail = 0;  
        x.ID = i;  
        H.Insert(x); //插入并更新小顶堆  
    }  
    for(int i=n; i>=1; i--){  
        H.DeleteMin(x); //从小顶堆中找到根结点x作为当前机器, 安排第i个任务  
        cout<<“将机器” <<x.ID <<“从” <<x.avail<<“到” <<(x.avail+a[i].time)  
            <<“的时间段分配给作业”<<a[i].ID<<endl;  
        x.avail +=a[i].time;  
        H.Insert(x); //将当前机器x重新插入并更新小顶堆  
    }  
}
```




例4. 多机调度问题-得到最优解的情况

- ▶ 设7个独立作业{1,2,3,4,5,6,7}由3台机器 M_1 , M_2 和 M_3 加工处理, 各作业所需的处理时间分别为{2,14,4,16,6,5,3}
- 按算法greedy产生的作业调度如下图所示, 所需的加工时间为17。





例4. 多机调度问题-得不到最优解的反例

- ▶ 设8个独立作业由3台机器 M_1 , M_2 和 M_3 加工处理, 各作业所需的处理时间分别为 $\{16, 14, 13, 11, 8, 6, 5, 3\}$
 - 按算法greedy产生的作业调度所需加工时间为27
 - 实际最优的作业调度所需加工时间为26

Greedy调度

机器 M_1 :	16, 6, 5	27
机器 M_2 :	14, 8, 3	25
机器 M_3 :	13, 11	24

最优调度

机器 M_1 :	16, 6, 3	25
机器 M_2 :	14, 11	25
机器 M_3 :	13, 8, 5	26



Next: 回溯法